PowerShell Scripting Language

What is PowerShell?

PowerShell is Microsoft's automation framework and command-line shell used to automate system tasks across **Windows, Linux, and macOS**.

Why Use It?

- Manual tasks like file, user, or system management are repetitive.
- PowerShell scripts automate these operations efficiently.
- **Example:** Install the same software on 100 computers using one PowerShell script instead of 100 manual steps.

PowerShell Overview

Component	Description
PowerShell Console	Command-line interface for executing cmdlets
PowerShell ISE	GUI-based scripting environment with debugging
Cmdlets	Built-in PowerShell commands (verb-noun format)
Scripts (.ps1)	Text files containing reusable PowerShell commands

Version Types

- **Windows PowerShell (v1–v5.1):** Windows-only.
- **PowerShell Core (v6+):** Cross-platform, open-source version.

PowerShell vs CMD vs Bash

Feature	PowerShell	CMD	Bash
Platform	Windows, Linux, macOS	Windows only	Linux, macOS, WSL
Output Type	Objects	Text	Text
Scripting File	`.ps1`	`.bat`	`.sh`
Use Case	Automation, DevOps	Legacy scripts	Linux administration

Example:

#Bash - text-based

ls -la | grep "txt"

#PowerShell - object-based

Get-ChildItem | Where-Object { $_.Name -like "*.txt" }

PowerShell ISE (Integrated Scripting Environment)

A graphical editor for creating, editing, and executing PowerShell scripts.

Why Use It?

- Code highlighting & auto-suggestions (IntelliSense)
- Debugging support (breakpoints, step execution)
- Multiple scripts open in one window

- Ideal for beginners and quick prototyping

Hands-On Lab:

```
Write-Host "Hello from PowerShell ISE!" -ForegroundColor Green
Get-Process | Select-Object -First 3
```

Run

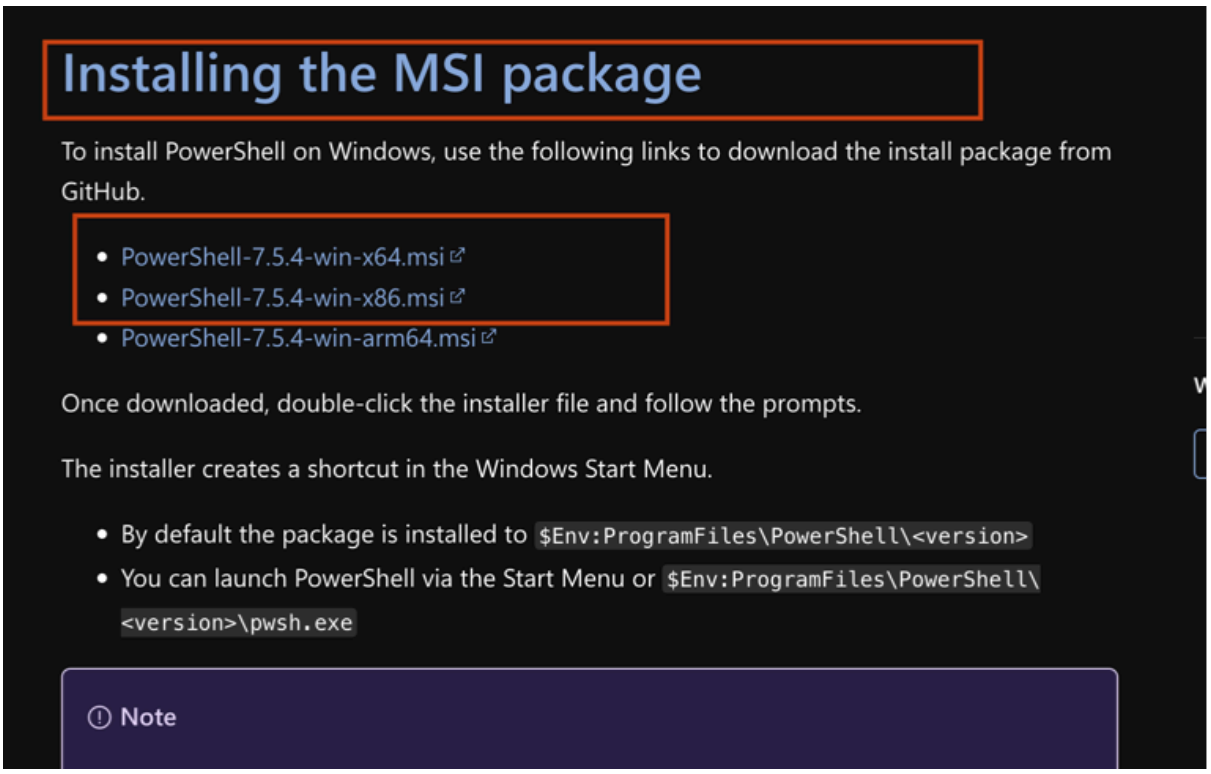
Environment Setup (Windows, Linux, macOS)

Windows

- PowerShell (built-in) → Open from Start Menu
- For PowerShell Core:
  Download from [GitHub PowerShell Releases]

-

Download PowerShell latest version :



Installing the MSI package

To install PowerShell on Windows, use the following links to download the install package from GitHub.

- PowerShell-7.5.4-win-x64.msi
- PowerShell-7.5.4-win-x86.msi
- PowerShell-7.5.4-win-arm64.msi

Once downloaded, double-click the installer file and follow the prompts.

The installer creates a shortcut in the Windows Start Menu.

- By default the package is installed to `$Env:ProgramFiles\PowerShell\<version>`
- You can launch PowerShell via the Start Menu or `$Env:ProgramFiles\PowerShell\<version>\pwsh.exe`

Note

Linux (Ubuntu)

sudo apt update
sudo apt install -y powershell
pwsh

macOS

brew install powershell

pwsh

Verification Command:

$PSVersionTable

PowerShell Installation & Setup Guide

Part A — Download the PowerShell Installer (Windows)

1. Locate the Official Installer

What it is:
The **MSI package** (Microsoft Installer) that installs PowerShell 7 on your Windows system.

Why it's used:
It's the most reliable and standardized way to install PowerShell — ensuring correct installation under
`C:\Program Files\PowerShell\7\`.

Steps:

- Open your browser and search for **"PowerShell 7 download"**.
- Visit the official **GitHub – PowerShell Releases** page.
- Download the latest **`PowerShell-7.x.x-win-x64.msi`** (for 64-bit systems).
  (Use x86 only for older 32-bit Windows — rare today.)

💡 **Tip:** GitHub hosts official Microsoft releases — secure, version-controlled, and always up to date.

Part B — Installing PowerShell Using the GUI (MSI)

1. **Run the MSI File**
   - Double-click `PowerShell-7.x.x-win-x64.msi`.
   - The MSI installer handles all setup tasks including Start Menu registration and environment setup.
2. **Follow the On-Screen Steps:**
   - Click **Next** on the welcome page.
   - Accept the license → click **Next**.
   - Keep the default path:
     `C:\Program Files\PowerShell\7\`

    - Click **Install**, wait for completion, then **Finish**.

3. **Installer Summary:**

    - Files are copied to `C:\Program Files\PowerShell\7\`.
    - Shortcuts are created in the Start Menu.
    - `pwsh` command may be added to your PATH automatically.

Part C — Verify Installation

1. Launch PowerShell 7

- Open **Start Menu** → type **pwsh** → press **Enter**,
  or open **Windows Terminal** and select the **PowerShell (pwsh)** profile.

2. Check Version

Inside `pwsh`, run:

```
$PSVersionTable
$PSVersionTable.PSVersion
```

Output:
Displays version details (e.g., `7.4.1`).

3. Confirm `pwsh` is Available from Any Terminal

Open **Command Prompt** or the legacy **Windows PowerShell (5.1)** and test:

```
pwsh -c "$PSVersionTable.PSVersion"
```

If you see **"command not found"**, your PATH wasn't set — fix it in the next step.

Part D — Add `pwsh` to PATH Manually (if needed)

1. Why?

Adding PowerShell to PATH allows you to invoke `pwsh` from **any** terminal or automation script.

2. Steps (Graphical Method):

1. Open **Start** → search "**Edit the system environment variables**" → open.
2. Click **Environment Variables...**
3. Under **System variables**, select **Path** → click **Edit**. Click **New** → add:
   C:\Program Files\PowerShell\7\
4. Click **OK** → close all dialogs.
5. Open a **new** terminal (changes apply to new sessions).
6. Run: pwsh -c "$PSVersionTable.PSVersion"

3. PowerShell Method (Admin):

```
$pwshPath = 'C:\Program Files\PowerShell\7'
[Environment]::SetEnvironmentVariable('Path', $env:Path + ';' + $pwshPath,
[EnvironmentVariableTarget]::Machine)
```

✅ Appends PowerShell's directory to the system PATH permanently.

Part E — Integrate with Visual Studio Code

1. Install VS Code

Download from https://code.visualstudio.com/ and install normally.

2. Install PowerShell Extension

- Open **VS Code** → press `Ctrl + Shift + X` → search **PowerShell (by Microsoft)** → click **Install**.

Why:
Provides syntax highlighting, IntelliSense, debugging, and integrated terminal experience.

3. Configure VS Code Terminal

Set your default shell:

File → Preferences → Settings → search "terminal.integrated.shell.windows"

Set the value to:
`C:\Program Files\PowerShell\7\pwsh.exe`

Part F — Uninstall PowerShell

1. Using Settings (GUI):

Go to **Settings** → **Apps** → **Installed Apps**, search **PowerShell 7**, and uninstall.

2. Using winget:

winget uninstall --id Microsoft.Powershell

3. Clean-up:

If you manually added the PATH, remove it under Environment Variables.

PowerShell in Visual Studio Code

Installation Steps:

1. Install VS Code → https://code.visualstudio.com
2. Open Extensions → Search **"PowerShell" by Microsoft**

Verify Installation

Hands-On Verification:

```
# Check PowerShell version
$PSVersionTable

# Test commands
Get-Date
Get-Location
Get-ChildItem
```

Get-Help Get-Process

Check available modules
Get-Module -ListAvailable | Select-Object -First 5

Expected Output (Example):

PSVersion 7.5.4
PSEdition Core
Platform Unix

Setup is successful if these commands execute without error.

Essential PowerShell Commands

Category	Example Cmdlets	Purpose
System Info	`Get-ComputerInfo`, `hostname`, `$PSVersionTable`	System details
Files & Folders	`Get-ChildItem`, `New-Item`, `Remove-Item`	File operations
Processes	`Get-Process`, `Start-Process`, `Stop-Process`	Process management
Network	`Get-NetIPAddress`, `Test-Connection`	Network diagnostics
Services	`Get-Service`, `Start-Service`	Manage services
Users	`Get-LocalUser`, `whoami`	User information

Category	Example Cmdlets	Purpose
System Info	`Get-ComputerInfo`, `hostname`, `$PSVersionTable`	System details
Files & Folders	`Get-ChildItem`, `New-Item`, `Remove-Item`	File operations

Processes	`Get-Process`, `Start-Process`, `Stop-Process`	Process management
Network	`Get-NetIPAddress`, `Test-Connection`	Network diagnostics
Services	`Get-Service`, `Start-Service`	Manage services
Users	`Get-LocalUser`, `whoami`	User information

Cmdlets all commands list:

https://learn.microsoft.com/en-gb/powershell/module/microsoft.powershell.management/?view=powershell-7.5

File Operations (Hands-On)

```
# List files
Get-ChildItem

# Create file and folder
New-Item -Name "test.txt" -ItemType "File"
New-Item -Name "test" -ItemType "Directory"

# Delete file/folder
Remove-Item "test.txt"
Remove-Item "test"

# Copy file
Copy-Item "1.var.ps1" "11.var.ps1"
```

Help System (Documentation & Assistance)

Get Help Anytime:

```
Get-Help Get-Process            # Basic
Get-Help Get-Process -Detailed   # Detailed info
Get-Help Get-Process -Examples   # Practical examples
Get-Help Get-Process -Online     # Open browser help
Update-Help                      # Refresh help content
```

Hands-On: Create First PowerShell Script

File: `hello.ps1`

```
# My First PowerShell Script
Write-Host "=== My First PowerShell Script ===" -ForegroundColor Cyan

$computerName = $env:COMPUTERNAME
$userName = $env:USERNAME
$currentTime = Get-Date

Write-Host "Computer Name: $computerName" -ForegroundColor Green
Write-Host "User Name: $userName" -ForegroundColor Yellow
Write-Host "Current Time: $currentTime" -ForegroundColor Magenta
Write-Host "`nFiles in current directory:" -ForegroundColor Cyan
Get-ChildItem | Select-Object -First 5 | Format-Table Name, Length, LastWriteTime
```

Output Example

```
=== My First PowerShell Script ===
Computer Name: DESKTOP-ABC123
User Name: Nahid
Current Time: Tuesday, Oct 28, 2025 12:30 PM
Files in current directory:
Name          Length LastWriteTime
----          ------ -------------
file1.txt       1024   10/28/2025 12:16
file2.ps1        140    10/28/2025 12:20
```

Summary

✅ **Key Takeaways**

- PowerShell = Automation + Scripting + Cross-platform control

- Cmdlets follow **Verb-Noun** convention (`Get-Process`, `Set-Location`)
- VS Code + PowerShell extension = Best scripting environment
- Use **ISE** for beginners and debugging
- Always verify the environment using `$PSVersionTable`

Variables, Data Types & Operators

Variables

A **variable** is simply a *container that stores data* in memory for reuse.

```
# Without variables - Hardcoded (❌ Bad Practice)
Connect-Service -Server "prod-db-01" -Port 1433 -Database "Customers"

# With variables - Configurable (✅ Best Practice)
$DatabaseServer = "prod-db-01"
$DatabasePort = 1433
$DatabaseName = "Customers"
Connect-Service -Server $DatabaseServer -Port $DatabasePort -Database $DatabaseName
```

Example:

```
$name = "xyz"
$age = 30
Write-Host "User: $name, Age: $age"
```

Key Point: Every variable in PowerShell begins with a **$** sign.

🔹 Data Types in PowerShell

PowerShell automatically detects data types, but you can also define them explicitly.

Data Type	Example	Description
String	`"Hello World"`	Text or characters
Integer	`42`	Whole number
Double	`3.14`	Decimal number
Boolean	`$true / $false`	True or False
DateTime	`Get-Date`	System date and time

Example:

```
$stringVar = "Hello"
```

```
$intVar = 42
$doubleVar = 3.14
$boolVar = $true
$dateVar = Get-Date
```

Operators in PowerShell

Operators perform actions like calculation, comparison, and logical evaluation.
They're the *decision-making backbone* of automation scripts.

1️⃣ Arithmetic Operators
2️⃣ Comparison Operators
3️⃣ Logical Operators

1️⃣ Arithmetic Operators

Used for mathematical calculations.

Operator	Meaning	Example	Output
`+`	Addition	`10 + 5`	`15`
`-`	Subtraction	`10 - 3`	`7`
`*`	Multiplication	`4 * 5`	`20`
`/`	Division	`10 / 2`	`5`
`%`	Modulus (Remainder)	`10 % 3`	`1`

Demo:

```
$a = 10; $b = 3
Write-Host "Sum: $($a + $b)"
Write-Host "Modulus: $($a % $b)"
```

2️⃣ Comparison Operators

Used to evaluate conditions or match patterns.

Operator	Meaning	Example	Result
`-eq`	Equal to	`5 -eq 5`	True
`-ne`	Not equal to	`5 -ne 3`	True
`-gt`	Greater than	`10 -gt 5`	True
`-lt`	Less than	`5 -lt 10`	True
`-like`	Wildcard pattern match	`"Admin01" -like "Admin*"`	True
`-match`	Regex match	`"error_log" -match "error"`	True

Example:

```
if ($username -like "Admin*") {
    Write-Host "Admin account detected!"
}
```

3️⃣ Logical Operators

Combine multiple conditions in automation scripts.

Operator	Description	Example	Output
`-and`	Logical AND	`(5 -gt 3) -and (10 -gt 5)`	True
`-or`	Logical OR	`(2 -lt 3) -or (5 -eq 10)`	True
`-not` / `!`	Logical NOT	`-not (5 -eq 10)`	True
`-xor`	Exclusive OR	`$true -xor $false`	True

Example:

```
$userEnabled = $true
$passwordExpired = $false

if ($userEnabled -and (-not $passwordExpired)) {
    Write-Host "Access granted "
}
```

Arrays

What is an Array?

An **array** is a collection of multiple values stored within a single variable.
It helps manage lists of data efficiently — like servers, ports, or usernames.

Examples

```
$fruits = @("Apple", "Banana", "Cherry")
$ports  = @(22, 80, 443, 3389)
$servers = @("web01", "db01", "api01")
```

Key Concepts

- `@()` → Defines an array
- `[0]` → Access value by index (0-based)
- `.Length` or `.Count` → Returns array size

Array Operations

```
$fruits += "Orange"   # Add item
$fruits = $fruits | Where-Object { $_ -ne "Banana" }  # Remove item

Write-Host "Number of ports: $($ports.Count)"
```

Use Case: Managing multiple servers, checking port lists, or user provisioning in bulk.

$_ = it represent the current items.

- Contains the current object in the pipeline object

-notin = its check not available in list
-in = its check available in the list

Conditional Statements

Conditional statements allow scripts to **make decisions dynamically** based on input values or system states.

- ☐ Conditional statements enable **decision-making** in scripts.
- ☐ They determine **which code path** to execute based on whether a condition evaluates to **True** or **False**.

Types of Conditional Statements

- `if` Statement
- `if-else` Statement

- `if-elseif-else` Statement
- `switch` Statement

1️⃣ IF Statement

The `if` statement checks a **single condition**.
If the condition evaluates to **True**, the code block inside executes.

Syntax:

```
if (condition) {
    # Code to execute if condition is TRUE
}
```

Example – Basic IF Statement:

```
$number = 10

if ($number -gt 5) {
    Write-Host "Number is greater than 5" -ForegroundColor Green
}
```

Explanation:

- `$number -gt 5` → compares if `$number` is greater than 5

- If True → prints message in green color
- If False → no action is taken

2️⃣ IF-ELSE Statement

`if-else` provides **two possible outcomes**:

- Executes one block when the condition is **True**
- Executes another when the condition is **False**

Syntax:

```
if (condition) {
    # Code if condition is TRUE
} else {
    # Code if condition is FALSE
}
```

---

3 IF-ELSEIF-ELSE Statement

This structure allows you to test **multiple conditions sequentially**.

- ☐ Once a condition evaluates to **True**, the rest are skipped.

Syntax:

```
if (condition1) {
    # Code for condition1 TRUE
} elseif (condition2) {
    # Code for condition2 TRUE
} elseif (condition3) {
    # Code for condition3 TRUE
} else {
    # Code if all conditions are FALSE
}
```

4 SWITCH Statement

The `switch` statement is a **cleaner alternative** to multiple `if-elseif` blocks.
It compares a single expression against **multiple possible values**.

Syntax:

```
switch (expression) {
    value1 { # Code for value1 }
    value2 { # Code for value2 }
    value3 { # Code for value3 }
    default { # Code if no match }
}
```

Real-Life Use Cases:

- `if` and `elseif`: Decision-making in automation scripts
- `switch`: Handling multiple service states, file types, or response codes
- **Used for:** Role assignment, permission control, or input validation.
- Conditional logic is essential for **infrastructure automation**, **alert systems**, and **DevOps pipelines**

Loops

A **loop** allows you to **repeat a block of code multiple times** — useful when you need to automate repetitive tasks efficiently.

Use Cases

- Scanning multiple log files for errors
- Automatically creating several user accounts
- Continuously checking availability of 100+ IPs

Loop Types in PowerShell

PowerShell provides **four main types of loops**:

1️⃣ `for` — Used for fixed number of iterations
2️⃣ `foreach` — Used to loop through items in a collection
3️⃣ `while` — Runs while a condition remains true
4️⃣ `do-while` — Similar to `while`, but executes the block at least once

---

1️⃣ FOR Loop

`for` loop is used when you already know how many times you want to repeat an operation.
It consists of **three parts**:

- **Initialization** → Starting point of the loop variable
- **Condition** → Defines when the loop should stop
- **Increment/Decrement** → Updates the loop variable each iteration

Syntax:

```
for (<Initialization>; <Condition>; <Increment>) {
    # Code to execute
}
```

Example:

```
for ($i = 1; $i -le 5; $i++) {
    Write-Host "Iteration number: $i"
}
```

Explanation:

- `$i = 1` → starts the counter
- `$i -le 5` → runs the loop while `$i` is less than or equal to 5
- `$i++` → increases `$i` by 1 after each iteration

Tip: Use `for` loops when the **number of repetitions is known** — for example, iterating through a fixed range of IDs, ports, or test cases.

Used for: Running a command fixed number of times.

2️⃣ FOREACH Loop

The `foreach` **loop** is used when you want to perform an operation on **each item within a collection or array**.

- ➢ It automatically accesses every element, one by one.
  - Ideal for iterating through a list of users, servers, files, or IP addresses.

Syntax:

```
foreach ($item in $collection) {
    # Process each $item
}
```

Example: Basic Foreach Loop

```
$users = @("Alice", "Bob", "Charlie")
foreach ($user in $users) {
    Write-Host "Creating user: $user"
}
```

Explanation:

- `$users` → defines an array (list of names)

- `$user in $users` → loops through each element in the array
- `Write-Host` → prints output for each item

Output:

Creating user: Alice
Creating user: Bob
Creating user: Charlie

Best Practice:

Use `foreach` when working with **collections, lists, or objects** — especially useful for automation scripts that manage multiple systems or files.

3️⃣ WHILE Loop

A **`while` loop** runs **as long as a given condition is true**.
Once the condition becomes false, the loop stops immediately.

- ☐ Think of it like a continuous monitoring process that stops when a threshold is reached.
- ☐ Executes **as long as the condition is true.**

Syntax:

```
while (<condition>) {
    # Code to execute while condition is true
}
```

Example: Basic While Loop

```
$count = 1
while ($count -le 3) {
    Write-Host "Checking logs... ($count)"
    $count++
}
```

Used for: Continuous system monitoring or status checks.

Explanation:

- `$count -le 3` → loop continues as long as this condition remains true
- `$count++` → increases the counter after each iteration

Output:

Checking system logs... (1)

Checking system logs... (2)

Checking system logs... (3)

Best Practice:

Use `while` loops when **you don't know the exact number of iterations**, such as:

- Waiting for a service to start
- Continuously checking system status
- Retrying an operation until success

---

4 DO-WHILE Loop

The **`do-while` loop** always executes **at least once**, even if the condition is false.
After the first execution, it checks the condition — if true, it repeats.

- ☐ In short: the code inside the loop runs **before** the condition check.

Syntax:

```
do {
    # Code to execute at least once
} while (<condition>)
```

Example:

```
$retry = 0
do {
    Write-Host "Trying connection... Attempt $retry"
    $retry++
} while ($retry -lt 3)
```

Explanation:

- The first attempt **always runs**, regardless of the condition
- After execution, the condition is checked (`$retry -lt 3`)
- If true → loop repeats; if false → stops

Output:

Trying connection... Attempt 0

Trying connection... Attempt 1

Trying connection... Attempt 2

Real-Life Example:

Use `do-while` in **retry mechanisms**, such as:

- Retrying failed API calls
- Network connection tests
- Checking service availability at intervals

Used for: Retry mechanisms in automation or connection tests.

PowerShell Pipelines

A **Pipeline (`|`)** allows the output of one command to be passed as input to another command.
It is one of PowerShell's **most powerful features** for automation and data handling.

Why Do We Use Pipelines?

- To **combine multiple commands** for advanced operations
- To **filter, sort, and manipulate** data seamlessly
- To **simplify complex administrative tasks**

Examples

Get-Process | Select-Object -First 3

Get-Process | Where-Object { $_.CPU -gt 100 }

Get-ChildItem | Where-Object { $_.Length -gt 10 }

Get-ChildItem | Where-Object { $_.Length -gt 10 } | Select-Object -First 5

Get-Process | Where-Object { $_.CPU -gt 100 } | Select-Object Name, CPU, ID | Format-Table -AutoSize

Explanation:

- `|` → passes data from one command to another
- `$_` → represents the current object in the pipeline
- `Where-Object` → filters output

- `Select-Object` → extracts specific properties

Use Cases

- Filtering high CPU processes
- Scanning large directories efficiently
- Extracting and formatting log or event data

HASHTABLES (KEY-VALUE STORE)

A **Hashtable** is a **key-value pair data structure** (similar to a **Python dictionary** or a **JSON object**).
It stores data in the form of `Key → Value`, enabling fast lookups based on keys.

- ☐ In PowerShell, a Hashtable is an **unordered collection** of key-value pairs.
- ☐ It is widely used in production scripts to store configurations, metadata, or JSON-like structures efficiently.

Why Use Hashtables?

- To **organize related data** under one variable
- To **store configuration settings** or metadata
- To **group structured information** like user details or system info

---

Example:

Key	Value
Name	John
Age	30
Role	DevOps Engineer

Syntax:

```
# Method 1 – Inline Declaration
$hashtable = @{
    Key1 = "Value1"
    Key2 = "Value2"
}

# Method 2 – Stepwise Assignment
$hashtable = @{}
$hashtable["Key1"] = "Value1"
$hashtable.Key2 = "Value2"
```

Explanation:

- `{}` → defines key-value pairs
- Access using `.` or `['key']` notation

Use Case:

- Store configuration data for multiple servers

- Maintain key-value mappings like `ServerName → IPAddress`
- Manage user data, environment variables, or credentials safely within scripts

---

Nested Hashtables

You can define **Hashtables inside Hashtables** for structured data.

```
$users = @{
    "user1" = @{Name = "XYZ"; Role = "DevOps"; Salary = 125000}
    "user2" = @{Name = "John"; Role = "Developer"; Salary = 80000}
    "user3" = @{Name = "Sarah"; Role = "Manager"; Salary = 90000}
    "user4" = @{Name = "Mike"; Role = "Admin"; Salary = 80000}
}


Write-Host "$($users.user1.Name)"
Write-Host "$($users.user2.Role)"
```

Used for:

- Storing **environment-wise configurations**
- Representing **server inventory mappings**

FUNCTIONS

A **Function** is a **reusable block of code** that performs a specific task. Instead of repeating code, you define it once and call it whenever required.

- ☐ Example: You can create a function named `Security-Scan` and call it whenever you need to run a scan — without rewriting the code.

Why Use Functions?

- Avoid repetitive code
- Keep scripts organized and readable
- Simplify debugging and maintenance
- Improve collaboration in team environments

🔹 **Syntax:**

```
function Function-Name {
    param (
        # Optional parameters
    )
    # Code block to execute
}
```

Example – Basic Function

```
function SayHello {
    Write-Host "Hello, welcome to PowerShell automation!"
}

# Calling the function
SayHello
```

Explanation:

- Function defined once using `function` keyword
- Can be invoked by calling its name (`Say-Hello`)
- Used for repetitive operations like log checks, backups, scans, or service restarts

Parameterized Functions

To make functions **dynamic and flexible** using parameters.

```
function Get-ServerStatus {
    param([string]$ServerName)
    Test-Connection -ComputerName $ServerName -Count 1
}

Get-ServerStatus -ServerName "10.0.0.1"
```

Explanation:

- `param()` → accepts user input
- `[string]` → defines the expected data type
- `$ServerName` → parameter variable
- `Test-Connection` → performs a ping test

Ping multiple servers in an environment monitoring script.

Returning Values from a Function

```
function Add-Numbers {
    param([int]$a, [int]$b)
    return ($a + $b)
}

$result = Add-Numbers -a 5 -b 3
Write-Host "Sum is: $result"
```

- `return` sends the computed result back
- `$result` stores function output for further processing

Use Case: Mathematical or statistical operations in automation workflows.

File System Operations (File I/O)

File I/O (Input/Output) refers to **creating, reading, writing, or deleting files and directories**.

Why Do We Use File Operations?

- ➢ To store and read log data
- ➢ To save scan or audit results
- ➢ To maintain configuration or evidence files
- ➢ To automate reporting and system monitoring

Common File Commands

Task	Command
List files	`Get-ChildItem`
Create file	`New-Item -ItemType File`
Create folder	`New-Item -ItemType Directory`
Copy file	`Copy-Item`
Move file	`Move-Item`
Delete file	`Remove-Item`
Read content	`Get-Content`
Write content	`Set-Content` / `Add-Content`

Create & Read Files

```
New-Item -Path "C:\Scripts\demo.txt" -ItemType File

Set-Content -Path "C:\Scripts\demo.txt" -Value "This is PowerShell training file."

Get-Content -Path "C:\Scripts\demo.txt"
```

Explanation

- `New-Item` → creates a new file
- `Set-Content` → writes or overwrites file content
- `Get-Content` → reads file content

Store automation results or script logs.

Append to File

```
Add-Content -Path "C:\Scripts\demo.txt" -Value "New log entry at $(Get-Date)"
```

Explanation:

- `Add-Content` → appends new lines without overwriting
- `$(Get-Date)` → adds current timestamp dynamically

Use Case: Append monitoring logs or security scan results to existing reports.

Reading Files Line by Line

```
Get-Content "C:\Scripts\servers.txt" | ForEach-Object {
    Write-Host "Connecting to: $_"
}
```

Explanation

- `Get-Content` reads file line by line
- `$_` represents the current line in the pipeline
- `ForEach-Object` executes code for each line

For example — Loop through a list of servers or IPs for health checks or patch automation.

ERROR HANDLING & STRUCTURED LOGGING

Error handling ensures that when a script encounters an error, it **does not crash abruptly** but instead responds **gracefully** with meaningful messages.

In PowerShell, error handling is implemented using the `try`, `catch`, **and** `finally` blocks.

Why Use Error Handling?

- Prevents script crashes
- Provides user-friendly error messages
- Makes debugging easier
- Helps develop **reliable and production-grade** automation scripts

---

Syntax:

```
try {
    # Code that might throw an error
}
catch {
    # Handle the error
    Write-Host "An error occurred: $($_.Exception.Message)" -ForegroundColor Red
}
finally {
    # Cleanup code (optional)
    Write-Host "Execution completed."
}
```

Explanation:

- `try` → executes main code
- `catch` → handles any exceptions
- `$_.Exception.Message` → returns the actual error message

- `finally` → runs cleanup actions regardless of success or failure

Example – Error Handling

```
try {
    Get-Content "C:\nonexistentfile.txt"
}
catch {
    Write-Host " File not found! Please check the path." -ForegroundColor Red
}
finally {
    Write-Host " Script execution finished." -ForegroundColor Green
}
```

Use Cases:

- Handling missing configuration files during deployment
- Validating API or network responses

- Logging system errors in DevOps monitoring pipelines

- Ensuring CI/CD pipeline continues safely even if one task fails

Try–Catch–Finally

```
function Test-FileAccess {
    param([string]$FilePath)

    try {
        Write-Host "Attempting to access: $FilePath" -ForegroundColor Yellow

        $content = Get-Content -Path $FilePath -ErrorAction Stop
        Write-Host "File accessed successfully!" -ForegroundColor Green
        return $content
    }
    catch {
        Write-Host " ERROR: Could not access file!" -ForegroundColor Red
        Write-Host "Details: $($_.Exception.Message)" -ForegroundColor Yellow
        return $null
    }
    finally {
        Write-Host " File access attempt completed.`n" -ForegroundColor Gray
    }
}

# Test Example
Test-FileAccess -FilePath "C:\Scripts\test.ps1"
Test-FileAccess -FilePath "C:\Scripts\missing.ps1"
```

Breakdown:

- **try** → attempt risky operation
- **catch** → handle failure scenarios
- **finally** → always executes (cleanup, logging, etc.)

Think of this as an “insurance policy” for your PowerShell scripts.

Structured Logging

Structured logging means writing log entries in an organized, timestamped format that can later be used for auditing, debugging, or analysis.

Logging Example

```
$logFile = "C:\Scripts\activity.log"

try {
    Get-Content "C:\data.txt" | Out-File $logFile -Append
    Write-Host "Successfully read data."
}
catch {
    $errorMsg = "$(Get-Date): Error reading file - $($_.Exception.Message)"
    Add-Content -Path $logFile -Value $errorMsg
}
```

Explanation:

- `Out-File` / `Add-Content` → append log entries
- `$(Get-Date)` → timestamps every log
- Helps you track what happened, when, and why

A security automation script that records every scan result or error for later audit review.

Production-Grade Example

```
function ParseLog {
    param([string]$FilePath)
    try {
        $content = Get-Content $FilePath
        $errors = $content | Select-String "ERROR"
        if ($errors) {
            Add-Content "C:\Scripts\error_report.txt" "$(Get-Date): Found error in $FilePath"
        }
    } catch {
        Add-Content "C:\Scripts\error_report.txt" "$(Get-Date): Failed to read $FilePath - $
        ($_.Exception.Message)"
    }
}

$logs = Get-ChildItem "C:\Logs" -Filter "*.log"
foreach ($log in $logs) {
    ParseLog -FilePath $log.FullName
}
```

Scenario:
Used in **Security Operations (SOC)** for daily log scanning — automatically parsing large system logs, identifying “ERROR” entries, and saving reports for the incident response team.